

Programming Techniques 2026–2027

Lecture 2: Fortran Types, Expressions, and Control Flow

Hannu Parviainen

Universidad de la Laguna

September 30, 2026

Example: Triangle Area (Module + Program)

```
module triangle_operations
  implicit none
contains
  function area(x, y, z)
    real :: area ! function type
    real, intent(in) :: x, y, z
    real :: theta, height
    theta = acos((x**2 + y**2 - z**2) / (2.0*x*y))
    height = x * sin(theta)
    area = 0.5 * y * height
  end function area
end module triangle_operations

program triangle
  use triangle_operations
  implicit none
  real :: a, b, c
  print *, 'Welcome, please enter the lengths of the 3 sides.'
  read *, a, b, c
  print *, "Triangle's area:", area(a, b, c)
end program triangle
```

Coding Style (Recommendations)

Conventions

- ▶ Always use `implicit none`.
- ▶ Write keywords and intrinsics in lower case; use descriptive names (`n_planets`, not `np`).
- ▶ Use the symbolic relational operators (`<`, `==`, `/=`), not the legacy `.lt.`, `.eq.`, `.ne..`
- ▶ Use small, consistent indentation (2–4 spaces); indent blocks and units.
- ▶ Include unit names in corresponding `end` statements.

Note

To fit code on slides, I have relaxed some conventions in teaching materials.

Source Form (Free Form)

Rules

- ▶ Up to 132 characters per line.
- ▶ `!` introduces a comment.
- ▶ `&` continues a logical line.
- ▶ `;` separates multiple statements on one line.
- ▶ Blanks are significant: they separate keywords and names (`integeri` is an error).

Example

```
a = 5.2; m_star = 1.0      ! two statements on one line
p_years = sqrt(a**3 &      ! one statement on two lines
            / m_star)
```

Intrinsic Types

Default Intrinsic Types

▶ Character

- ▶ `character :: spectral_class`
- ▶ `character(len=12) :: star_name`

▶ Logical

- ▶ `logical :: is_binary`

▶ Numeric

- ▶ `integer :: n_planets`
- ▶ `real :: mass`
- ▶ `complex :: visibility`

Literal Constants

Examples

- ▶ Integers: 4096
- ▶ Reals: 1.0, 1.989e30, -0.5
- ▶ Logical: `.false.`, `.true.`
- ▶ Character: `"Barnard's Star"`, `'Barnard's Star'` (escaped quote)

Notes

- ▶ Only two logical values.
- ▶ Reals contain a decimal point (and may use exponential form).
- ▶ Character literals may be delimited by single or double quotes.

Implicit Typing (Turn It Off)

Rule

Without declarations, variables default to: `integer` if starting with `I-N`; otherwise `real`. This is error-prone (or, more plainly, stupid).

Best Practice

Always put `implicit none` immediately after any use statements.

Declarations (Numeric and Logical)

General Syntax

`<type> [, <attributes>] :: <vars> [= <value>]`

Examples

```
real :: mass
integer :: i, j
logical :: converged
real, dimension(3) :: position, velocity
integer :: n_steps = 1000
```

Arrays

`dimension(3)` declares a three-element array (more on arrays in Lecture 3).

Symbolic Constants (Parameters)

Definition

Symbolic constants are declared with the `parameter` attribute.

Examples

```
real, parameter :: pi = 3.14159
real, parameter :: G = 6.674e-11    ! m^3 kg^-1 s^-2
character(len=*), parameter :: host = 'Kepler-452', planet = "Kepler-452 b"
```

When to Use

- ▶ A variable should take only one value.
- ▶ To avoid “magic numbers”.
- ▶ For maintainability: values may change later.

Initialisation

Rules

Initial values may use literals or other parameters.

Examples

```
real :: distance, luminosity = 3.828e26
integer :: n_stars = 5, n_steps = 100
character(len=1) :: band = 'V'
character(len=10) :: star = 'Vega'    ! right-padded with blanks
logical :: converged = .false., verbose = .true.
real, parameter :: pi = 3.141592, au = 1.496e11, year = 3.156e7
real :: v_earth = 2 * pi * au / year    ! Earth's orbital speed (m/s)
```

Notes

A subset of intrinsics are allowed in initialisation expressions (e.g., kind, len, huge, tiny, epsilon, etc.).

Expressions

Types

Expressions can be numeric (e.g., `2*pi*r`), character (`star // ' b'`), or logical (`is_binary` and `converged`).

Use

Valid wherever an expression of the appropriate type is expected.

Assignment

Rule

The left-hand side is a variable; the right-hand side is an expression. The expression is evaluated first and then converted to the type of the variable. Numeric conversion happens silently.

Examples

```
m_total = m1 + m2
x = r * cos(phi)           ! cos uses radians
planet_name = star_name // ' b'
in_hz = (a >= a_inner .and. a <= a_outer)  ! in the habitable zone?
n_stars = 3.7              ! integer n_stars: truncated to 3
```

Intrinsic Numeric Operations

Operators

- ▶ `**` exponentiation (right-associative): `10**2`
- ▶ `*`, `/` multiplication, division: `10*7/4`
- ▶ `+`, `-` unary/binary plus/minus: `10+7-4`, `-3`

Notes

Operators apply to literals, scalars, and arrays. With arrays they act element by element, and array operands must have the same shape.

Relational Operators

Syntax

<, <=, ==, /=, >=, > (legacy forms: .lt., .le., .eq., .ne., .ge., .gt.)

Note that “not equal” is /=. Writing != would start a comment.

Example

```
is_giant = (radius > 10.0)
if (n_planets == 0) print *, 'No known planets'
```

Reals

Avoid exact equality with reals; compare within a tolerance:

```
real :: tol
tol = 1.0e-4
if (abs(m1 - m2) < tol) equal_mass = .true.
```

Logical Operators

Operators

- ▶ `.not.` — logical negation
- ▶ `.and.` — conjunction
- ▶ `.or.` — disjunction
- ▶ `.eqv.` — equivalence (same truth value)
- ▶ `.neqv.` — non-equivalence (different truth value)

Truth Table Highlights

- ▶ `.not.` `.true.` is `.false.`
- ▶ `.true.` `.and.` `.false.` is `.false.`
- ▶ `.true.` `.or.` `.false.` is `.true.`

Control Flow

Overview

Control constructs allow the normal sequential order of execution to be changed. Fortran supports:

- ▶ Conditional execution: `if, if ... then ... else, select case`
- ▶ Loops: `do, do while`
- ▶ Loop control: `exit, cycle`

if Statement

Syntax

`if (logical-expression) exec-stmt`

Example

```
if (flux > flux_max) flux_max = flux
if (b_mag - v_mag > 1.0) is_red = .true.
if (n_obs > 0) mean_flux = total_flux / n_obs
```

Block if

Example

```
if (e < 1.0) then
  print *, "Elliptical orbit (bound)"
else if (e > 1.0) then
  print *, "Hyperbolic orbit (unbound)"
else
  print *, "Parabolic orbit"
end if
```

Notes

Both `else` and `else if` are optional. Indentation improves readability.

select case

Example

```
select case (n_planets)
case (0)
  print *, "No known planets"
case (1)
  print *, "Single-planet system"
case (2:7)
  print *, "Multi-planet system"
case (8:)
  print *, "At least as many planets as the Solar System"
case default
  print *, "Invalid planet count"
end select
```

Notes

Cleaner and often more efficient than nested ifs.

Indexed do Loops

Example

```
do i = 1, 100, 1
  ...
end do
```

Syntax

`do var = start, stop [, step]` with step defaulting to 1.

Loop Variations

Examples

```
do i = 1, 30, 2      ! 15 iterations
do j = 30, 1, -2     ! countdown
do k = 30, 1, 2      ! zero-trip (skipped)
do l = 1, 30         ! step default = 1
```

Conditional exit

Example

```
frame = 0
do
  frame = frame + 1
  if (frame > 100) exit
  print *, "Reducing frame", frame
end do
print *, "Loop finished. frame now equals", frame
```

Explanation

A do without loop control repeats forever. `exit` jumps out of the current do loop.

Conditional cycle

Example

```
frame = 0
do
  frame = frame + 1
  if (frame >= 50 .and. frame <= 59) cycle    ! clouds: skip these frames
  if (frame > 100) exit
  print *, "Reducing frame", frame
end do
print *, "Loop finished. frame now equals", frame
```

Explanation

cycle skips to the next iteration of the innermost loop.

do while Loop

Example

```
do while (.not. converged)
  ...
end do
```

Equivalent

```
do
  if (converged) exit
  ...
end do
```

Named and Nested Loops

Example: counting bright pixels in an image

```
rows: do j = 1, ny
  cols: do i = 1, nx
    if (image(i, j) < 0.0) cycle rows      ! bad pixel: skip the rest of this row
    if (image(i, j) > saturation) exit rows ! saturated pixel: stop searching
    if (image(i, j) < threshold) cycle    ! same as cycle cols
    n_bright = n_bright + 1
  end do cols
end do rows
```

Notes

`exit` and `cycle` can name the loop they act on. Without a name, they act on the innermost loop.

Takeaways

Essentials

- ▶ Use `implicit none`; declare all variables.
- ▶ Prefer symbolic constants for fixed values.
- ▶ Write keywords in lower case and use the symbolic operators (`<`, `==`, `/=`).
- ▶ Control flow constructs: `if`, `do`, `do while`, `select case`.

Exercises

Instructions

- ▶ Write each exercise as a separate program in `students/<initials>/sandbox/102/`.
- ▶ Compile with checks on: `gfortran -Wall -fcheck=all -o prog prog.f90`
- ▶ The programs do not read input yet: set the input values with assignments at the start of the program. To test another case (including invalid values), edit them and recompile.
- ▶ Commit your programs and push them to your fork.

Overview

1. Kepler's third law (arithmetic, conversion)
2. Spectral classes (`select case`)
3. Habitable zone (logical operators)
4. Wien's law (indexed `do`)
5. Kepler's equation (`do while`)

Exercise 1: Kepler's Third Law

Task

Set the semi-major axis a (AU) of a planet and the mass M ($M_{\odot}$) of its star in variables. Print the orbital period in years, in days, and as a whole number of days.

- ▶ Store the number of days in a year (365.25) as a parameter.
- ▶ Stop with an error message if either value is not positive (stop 'message').
- ▶ Get the whole days by converting the period to an integer. Try both plain assignment and `int()`: what does -Wall say?

Physics

$P = \sqrt{a^3/M}$, with P in years, a in AU, and M in solar masses.

Test

Jupiter: $a = 5.2$, $M = 1.0 \rightarrow P \approx 11.86$ years ≈ 4331 days.

Exercise 2: Spectral Classification

Task

Set the effective temperature T_{eff} (K) of a star in an `integer` variable and print its spectral class using `select case`. Store the class letter in a `character(len=1)` variable and print it with `//`. Temperatures outside the sequence should give a message.

Physics

O	B	A	F	G	K	M
$\geq 30\,000$	10 000–29 999	7500–9999	6000–7499	5200–5999	3700–5199	2400–3699

Test

Sun: 5772 K $\rightarrow$ G. Vega: 9602 K $\rightarrow$ A. 1500 K $\rightarrow$ outside the sequence.

Exercise 3: Habitable Zone

Task

Set the luminosity L ($L_{\odot}$) of a star, and the semi-major axis a (AU) and eccentricity e of a planet in variables. Report whether the planet is *always*, *partly*, or *never* in the habitable zone (and if never, whether it is too hot or too cold).

- ▶ Use two logical variables: is the periapsis in the zone, and is the apoapsis in the zone?
- ▶ Hint: `.neqv.` is true when exactly one of its operands is true.
- ▶ Reject eccentricities outside $0 \leq e < 1$. What if the orbit crosses the whole zone?

Physics

Zone edges $r_{\text{in}} = \sqrt{L/1.1}$ and $r_{\text{out}} = \sqrt{L/0.53}$ AU; periapsis $a(1 - e)$, apoapsis $a(1 + e)$.

Test ($L = 1$)

Earth (1.0, 0.017): always. (1.3, 0.1): partly.

Venus (0.723, 0.007): too hot. Mars (1.524, 0.093): too cold.

Exercise 4: Wien's Law Table

Task

Print a table of the peak emission wavelength $\lambda_{\max}$ of a blackbody for $T = 3000, 6000, \dots, 30\,000$ K, using an integer loop variable. Then print the table again, hottest first, using a negative step.

- ▶ Print the loop variable after each loop. Can you explain the values?
- ▶ What would you get if b were the integer constant 2897772?

Physics

$\lambda_{\max} = b/T$, with $b = 2.897772 \times 10^6$ nm K.

Test

3000 K $\rightarrow$ 966 nm (infrared), 6000 K $\rightarrow$ 483 nm (visible), 30 000 K $\rightarrow$ 96.6 nm (ultraviolet).

Exercise 5: Kepler's Equation

Task

Set the mean anomaly M (rad) and eccentricity e of an orbit in variables, and solve Kepler's equation for the eccentric anomaly E using Newton's method.

- ▶ Start from $E_0 = M$ and iterate in a `do while` loop until $|E_{n+1} - E_n| < 10^{-6}$ or 50 iterations have been done.
- ▶ Use a logical variable `converged`. Print E and the number of iterations, or a warning if the iteration did not converge.

Physics

$$E - e \sin E = M, \quad E_{n+1} = E_n - \frac{E_n - e \sin E_n - M}{1 - e \cos E_n}$$

Test

$$M = 1.0, e = 0.5 \rightarrow E \approx 1.4987. \quad M = 1.0, e = 0.9 \rightarrow E \approx 1.8621.$$